

Assignment 3: HMM POS Tagger

December 12, 2025

1 Deriving the Viterbi Algorithm

1.1

$$P(t|w) = \frac{P(t, w)}{P(w)}$$

Since $P(w)$ is constant for all tag sequences maximizing $P(t|w)$ is equivalent to maximizing $P(t, w)$.

Since the logarithm is a strictly increasing function we can say that

$$\arg \max_t P(t|w) = \arg \max_t P(t, w) = \arg \max_t \log P(t, w)$$

We use the log transformation to convert products into sums, preventing numerical underflow when multiplying many small probabilities.

1.2

We can compute $\pi_j(t_j)$ recursively using:

$$\pi_j(t_j) = \max_{t_{j-1}} [\pi_{j-1}(t_{j-1}) + \text{score}(w, j, t_j, t_{j-1})]$$

Proof: By definition:

$$\pi_j(t_j) = \max_{t_1, \dots, t_{j-1}} \sum_{i=1}^j \text{score}(w, i, t_i, t_{i-1})$$

We can split this sum into two parts:

$$\pi_j(t_j) = \max_{t_1, \dots, t_{j-1}} \left[\sum_{i=1}^{j-1} \text{score}(w, i, t_i, t_{i-1}) + \text{score}(w, j, t_j, t_{j-1}) \right]$$

Since the score at position j only depends on t_{j-1} and t_j (not on earlier tags), we can first maximize over $t_1, \dots, t_{j-2}$ (giving $\pi_{j-1}(t_{j-1})$), then maximize over t_{j-1} :

$$\pi_j(t_j) = \max_{t_{j-1}} [\pi_{j-1}(t_{j-1}) + \text{score}(w, j, t_j, t_{j-1})]$$

1.3

-
- 1: For each tag t :
 - 2: $\pi_1(t) = \log P(t|\text{START}) + \log P(w_1|t)$
 - 3: For $j = 2$ to n , for each tag t_j :
 - 4: $\pi_j(t_j) = \max_{t_{j-1}} [\pi_{j-1}(t_{j-1}) + \log P(t_j|t_{j-1}) + \log P(w_j|t_j)]$
 - 5: $\text{bp}_j(t_j) = \arg \max_{t_{j-1}} [\pi_{j-1}(t_{j-1}) + \log P(t_j|t_{j-1})]$
 - 6: $t_n^* = \arg \max_t [\pi_n(t) + \log P(\text{STOP}|t)]$
 - 7: For $j = n - 1$ down to 1: $t_j^* = \text{bp}_{j+1}(t_{j+1}^*)$
-

Time Complexity: $O(n \times T^2)$ where n is the sentence length and T is the number of tags. At each position j (n positions), we compute $\pi_j(t)$ for each of T tags, and each computation requires maximizing over T previous tags.

1.4

The recurrence generalizes to any semiring $\mathcal{S} = \langle A, \otimes, \oplus, \mathbf{0}, \mathbf{1} \rangle$:

$$\pi_j(t_j) = \bigoplus_{t_{j-1}} [\pi_{j-1}(t_{j-1}) \otimes \text{score}(w, j, t_j, t_{j-1})]$$

Semiring properties used:

- **Associativity of $\otimes$:** Allows reordering of score multiplications
- **Distributivity of $\otimes$ over $\oplus$:** $a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$, which allows factoring out common terms
- **Identity elements:** $\mathbf{0}$ as identity for $\oplus$, $\mathbf{1}$ as identity for $\otimes$

For the **Viterbi semiring** $(\mathbb{R} \cup \{-\infty\}, \max, +, -\infty, 0)$:

- $\oplus = \max$ (generalized addition)
- $\otimes = +$ (generalized multiplication)

You would have to replace summation with max, and products with sums (in log space).

2 HMM Implementation

The HMM is implemented with the following components:

- **Transition probabilities:** $P(t_i|t_{i-1})$ estimated from bigram tag counts with add- α smoothing:

$$P(t_j|t_i) = \frac{\text{count}(t_i, t_j) + \alpha}{\text{count}(t_i) + \alpha \cdot |T|}$$

- **Emission probabilities:** $P(w|t)$ estimated from (word, tag) counts with add- α smoothing:

$$P(w|t) = \frac{\text{count}(t, w) + \alpha}{\text{count}(t) + \alpha \cdot |V|}$$

- **Special tokens:** $\langle \text{START} \rangle$ and $\langle \text{STOP} \rangle$ tokens added to each sentence for boundary modeling

3 Viterbi Decoding

The Viterbi decoder operates entirely in log space to avoid numerical underflow and uses smoothing for unknown words

4 Evaluation Results

4.1 Overall Accuracy

True Tag	Predicted	Count
NN	JJ	419
VBD	VBN	366
RB	IN	305
IN	RB	303
VBN	JJ	271
VBN	VBD	238
NN	NNP	224
JJ	NN	209
NNP	JJ	184
VBP	VB	162
VBG	NN	159
NN	NNS	149
IN	RP	137
NNPS	NNP	135
JJ	RB	131

Figure 1: Results

4.2 Hyperparameter Tuning

True Tag	Predicted	Count
NN	JJ	419
VBD	VBN	366
RB	IN	305
IN	RB	303
VBN	JJ	271
VBN	VBD	238
NN	NNP	224
JJ	NN	209
NNP	JJ	184
VBP	VB	162
VBG	NN	159
NN	NNS	149
IN	RP	137
NNPS	NNP	135
JJ	RB	131

Figure 2: Hyperparameters

4.3 Confusion Matrix Analysis

True Tag	Predicted	Count
NN	JJ	419
VBD	VBN	366
RB	IN	305
IN	RB	303
VBN	JJ	271
VBN	VBD	238
NN	NNP	224
JJ	NN	209
NNP	JJ	184
VBP	VB	162
VBG	NN	159
NN	NNS	149
IN	RP	137
NNPS	NNP	135
JJ	RB	131

Figure 3: Confusion Matrix

4.4 HMM vs. Baseline Comparison

The HMM tagger outperforms the baseline by approximately 2–3% accuracy.

4.5 Potential Improvements

Performance Improvements:

1. **Trigram HMM:** Use $P(t_i|t_{i-2}, t_{i-1})$ for better context modeling
2. **Morphological features:** Use word suffixes (-ing, -ed, -ly) to handle unknown words better

Speed Improvements:

1. **Tag pruning:** Only consider tags that can emit the current word
2. **Beam search:** Maintain only top- k tag sequences instead of all T possibilities